



Video Services Forum (VSF) Technical Recommendation TR-12

Client Device Discovery (CDD)

Bridging the First-Mile Gap: Modernizing Connectivity to the Cloud

April 30, 2026

VSF_TR-12_2026_04_30

INTELLECTUAL PROPERTY RIGHTS

THIS RECOMMENDATION IS BEING OFFERED WITHOUT ANY WARRANTY WHATSOEVER, AND IN PARTICULAR, ANY WARRANTY OF NONINFRINGEMENT IS EXPRESSLY DISCLAIMED. ANY USE OF THIS RECOMMENDATION SHALL BE MADE ENTIRELY AT THE IMPLEMENTER'S OWN RISK, AND NEITHER THE FORUM, NOR ANY OF ITS MEMBERS OR SUBMITTERS, SHALL HAVE ANY LIABILITY WHATSOEVER TO ANY IMPLEMENTER OR THIRD PARTY FOR ANY DAMAGES OF ANY NATURE WHATSOEVER, DIRECTLY OR INDIRECTLY, ARISING FROM THE USE OF THIS RECOMMENDATION.

LIMITATION OF LIABILITY

VSF SHALL NOT BE LIABLE FOR ANY AND ALL DAMAGES, DIRECT OR INDIRECT, ARISING FROM OR RELATING TO ANY USE OF THE CONTENTS CONTAINED HEREIN, INCLUDING WITHOUT LIMITATION ANY AND ALL INDIRECT, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES (INCLUDING DAMAGES FOR LOSS OF BUSINESS, LOSS OF PROFITS, LITIGATION, OR THE LIKE), WHETHER BASED UPON BREACH OF CONTRACT, BREACH OF WARRANTY, TORT (INCLUDING NEGLIGENCE), PRODUCT LIABILITY OR OTHERWISE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. THE FOREGOING NEGATION OF DAMAGES IS A FUNDAMENTAL ELEMENT OF THE USE OF THE

© 2025 Video Services Forum

This work is licensed under the Creative Commons Attribution-NoDerivatives 4.0 International License. To view a copy of this license, visit [HTTPS://creativecommons.org/licenses/by-nd/4.0/](https://creativecommons.org/licenses/by-nd/4.0/) or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

<http://www.videoservicesforum.org>

CONTENTS HEREOF, AND THESE CONTENTS WOULD NOT BE PUBLISHED BY VSF WITHOUT SUCH LIMITATIONS.



Executive Summary

Effectively utilizing the public cloud is essential for media organizations seeking to build dynamic media architectures. However, operators frequently report that the most challenging aspect of cloud workflows is accessing distributed, often ground-based sources and destinations, and establishing the initial live signal connection. While robust transport protocols like SRT, RIST, and TR-07 solve the physical streaming challenge, they still require manual IP management and complex navigation of firewalls, Network Address Translation (NAT), and security groups on both the sender and receiver. The full benefit of these protocols is only realized when the cloud operator's 24/7 global access extends to ground-based or mobile endpoints, regardless of their location behind corporate firewalls, VPNs, or the open internet. The Client Device Discovery (CDD) Technical Recommendation (TR-12) describes a method for device discovery, authentication, monitoring, and connection management via scalable, cloud-first techniques. Once a TR-12-enabled device is registered with a cloud service through a simplified pairing process, it remains persistently and globally accessible. This allows cloud operators or automated workflows to perform remote connection management without manual intervention at the edge.

TR-12 defines a client-server API and provides an open-source client SDK, an Application Reference Design (ARD), and a functional cloud service to serve as a remote endpoint. Recipients of this document are invited to download and test the SDK and submit technical comments.

The VSF also requests that recipients notify us of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the Recommendation set forth in this document, and to provide supporting documentation.

Table of Contents

Table of Contents.....	4
1 Introduction (Informative)	6
1.1 Contributors	7
1.2 About the Video Services Forum.....	7
2 Conformance Notation.....	7
3 References.....	8
4 Overview (Informative)	8
5 Host Configuration.....	9
6 Pairing.....	10
6.1 The pairing process	10
6.2 Throttling	12
6.3 Pairing Models	12
7 MQTT Communication	12
7.1 Overview (Informative)	12
7.2 MQTT Client: Settings	12
7.3 MQTT Client: Processing a Configuration change.	13
7.4 MQTT Client: Registration.....	14
7.5 MQTT Client: Reporting Status.....	15
7.6 MQTT Client: Certificate Rotation.....	15
7.7 MQTT Client: Deprovision.....	15
8 Subscriptions.....	16
8.1 Overview (informative)	16
8.2 Thumbnail Subscription.....	16
9 Nominal Operation Overview (informative)	17
9.1 Connect and Respond Loop (Informative)	17
9.2 Subscription Overview (Informative).....	17
9.3 Lifecycle Deprovision Overview (Informative)	18
10 The TR-12 Model (informative)	19

10.1	The TR-12 Model Version.....	19
10.2	The TR-12 Models.....	20
10.3	Registration Model.....	21
10.4	Configuration Model.....	22
10.5	Status Model	22
Appendix A	SMITHY Models:	22

Draft

1 Introduction (Informative)

Operators of cloud video workflows benefit from global resource access, 24/7 monitoring, and consistent programmatic APIs that enable automation across large topologies. However, these capabilities generally do not extend to on-premises or mobile source/destination systems. While a handful of device manufacturers offer proprietary remote access, there is no industry standard that allows scaled cloud video services to interoperate with these devices across the internet without custom, system-by-system integration. For most manufacturers, developing and maintaining an enterprise-grade, secure, and reliable remote access platform is neither feasible nor easily monetizable through hardware sales alone.

The success of any improvement requires addressing the needs of three primary participants:

- **Device Manufacturers:** Require the freedom to expose unique device controls with low barriers to entry and high resilience.
- **Broadcasters and Operators:** Are unlikely to adopt solutions that require replacing existing device fleets. Solutions available via over-the air (OTA) updates are preferred.
- **Cloud Infrastructure Providers:** Prefer investing in standards rather than device-specific customizations.

A viable solution must conform to cloud resource standards, including lifecycle patterns and host-provided, rotatable credentials. It must also utilize formal models based on modern API standards that offer built-in validation and inherent scalability, ensuring a consistent user experience across all device types with zero device-specific customization required by the host service.

This Technical Recommendation (TR-12) describes a methodology to achieve these goals by:

- **Providing SMITHY and OpenAPI Models:** Utilizing models ubiquitous in enterprise cloud services to auto-generate classes for requests, responses, accessors, and validation across most programming languages.
- **Defining Three Primary API Models: Registration, Status, and Configuration.** These allow manufacturers to define device and channel-level settings unique to their hardware. While the models enforce structure, they do not dictate specific settings, allowing both client and host to consume shared definitions.
- **Enabling Dynamic UI Generation:** Allowing the host service to dynamically generate a console UI with type-specific widgets for each parameter based solely on the client-provided *registration*. Configuration updates can be validated by the host service before transmission to the client.
- **Standardizing HTTPS and MQTT Transport:** Utilizing an OAuth-based pairing process with a simple code to grant service-provided authentication and persistent credentials. These credentials allow the authorized client to establish a low-latency, bidirectional, and encrypted MQTT connection to the host.
- **Establishing MQTT Topic Structures:** Defining the specific topics used by the TR-12 client and host to manage status, configuration, and lifecycle events.

1.1 Contributors

Thank you to the following individuals participated in the Video Services Forum TR-12 working group that help to shape this technical recommendation and provide feedback.

Mike Cronk: TVU Networks
Jeremy Gerdes: Osprey
Video
Sydney Lovely: AWS
Elemental
Drew Martin: Riedel

Michael Henry: AWS
Elemental
Jesus Oliva: Haivision
Jarrod Nix: Osprey Video
Rob Porter: Sony

Wes Simpson:
LearnIPvideo.com
Scott Whitcomb: Osprey
Video
Joel Williams: Fox

1.2 About the Video Services Forum

The Video Services Forum, Inc. (www.vsf.tv) is an international association dedicated to video transport technologies, interoperability, quality metrics and education. The VSF is composed of service providers, users and manufacturers. The organization's activities include:

- providing forums to identify issues involving the development, engineering, installation, testing and maintenance of audio and video services.
- exchanging non-proprietary information to promote the development of video transport service technology and to foster resolution of issues common to the video services industry.
- identification of video services applications and educational services utilizing video transport services.
- promoting interoperability and encouraging technical standards for national and international standards bodies.

The VSF is an association incorporated under the Not For Profit Corporation Law of the State of New York. Membership is open to businesses, public sector organizations and individuals worldwide. For more information on the Video Services Forum or this document, please e-mail opsmgr@vsf.tv.

2 Conformance Notation

Normative text is text that describes elements of the design that are indispensable or contains the conformance language keywords: "shall", "should", or "may". Informative text is text that is potentially helpful to the user, but not indispensable, and can be removed, changed, or added editorially without affecting interoperability. Informative text does not contain any conformance keywords.

All text in this document is, by default, normative, except the Introduction and any section explicitly labeled as "Informative" or individual paragraphs that start with "Note:"

The keywords "shall" and "shall not" indicate requirements strictly to be followed in order to conform to the document and from which no deviation is permitted.

The keywords "should" and "should not" indicate that, among several possibilities, one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required; or that (in the negative form) a certain possibility or course of action is deprecated but not prohibited.

The keywords "may" and "need not" indicate courses of action permissible within the limits of the document.

The keyword "reserved" indicates a provision that is not defined at this time, shall not be used, and may be defined in the future. The keyword "forbidden" indicates "reserved" and in addition indicates that the provision will never be defined in the future.

A conformant implementation according to this document is one that includes all mandatory provisions ("shall") and, if implemented, all recommended provisions ("should") as described. A conformant implementation need not implement optional provisions ("may") and need not implement them as described.

Unless otherwise specified, the order of precedence of the types of normative information in this document shall be as follows: Normative prose shall be the authoritative definition; Tables shall be next; followed by formal languages; then figures; and then any other language forms.

3 References

OAuth2: [HTTPS://oauth.net/2/](https://oauth.net/2/)

MQTT: [HTTPS://mqtt.org/](https://mqtt.org/)

SMITHY: [HTTPS://smithy.io/](https://smithy.io/)

SMITHY code generation: [HTTPS://smithy.io/2.0/guides/using-code-generation/index.html](https://smithy.io/2.0/guides/using-code-generation/index.html)

Any mention of references throughout the REST of this document refers to the versions described here, unless explicitly stated otherwise.

4 Overview (Informative)

The TR-12 protocol is comprised of two distinct communication components: an **HTTPS**-based protocol for initial pairing and large payloads, and an **MQTT**-based protocol for message communication. Client devices initiate both HTTPS requests and MQTT connections. Both require only outbound access via Port 443 to reach a public TR-12 service endpoint **over the internet** and to complete all necessary transactions. As a result, the client and host need not share a network, VPN, or require the host have prior knowledge of the client's private or public IP address. This "outbound-only", Port 443 architecture effectively bypasses the need for complex firewall rules or VPN overhead, making TR-12 an ideal solution for connecting devices located behind restrictive corporate or field networks to a public cloud (or privately hosted) service.

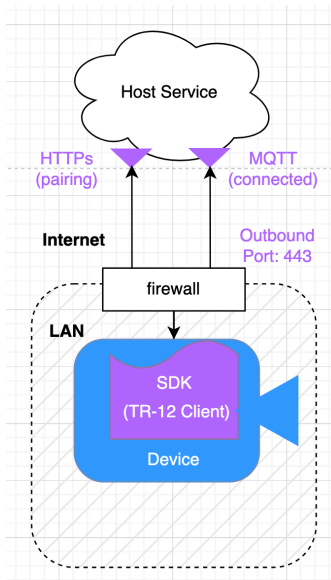


Fig1 shows a typical TR-12 system topology. A video streaming device within a corporate LAN/Firewall is running an instance of a **TR-12 Client SDK**. (<https://github.com/vsf-tv/gccg-cdd>). Here, the Client SDK runs as a stand-alone process and exposes a REST API on localhost. The device (a camera in this example) simply makes requests on the client via convenient APIs like *connect()*, *get configuration()*, *disconnect()*, etc. The Client SDK, in turn, handles all communication with the Host Service. *Note: TR-12 doesn't not require use of the open-source client SDK: Provided a client interacts with the host via the TR-12 protocol describe in this document, any client-side internal architecture is permitted.*

TR-12 host service's internal implementation is out of scope.

TR-12 imposes few requirements on the *host service* other than the successful implementation of the TR-12 pairing and device-facing MQTT API transactions. The internal host service architecture—specifically how a provider implements backend access to registered clients—is outside the scope of this document. In practice, host services will provide a centralized registry for device accessibility, monitoring, and configuration. These services must communicate with clients according to the TR-12 HTTP and MQTT models defined in the SMITHY specifications found in the Appendices.

Models:

The TR-12 protocol is defined primarily through API request/response models defined in the Appendix: SMITHY definition. Important API names and SMITHY model field names are referenced throughout this document, highlighted via italics such as *GetPairingCode* API and *AuthResponse Model*.

5 Host Configuration

TR-12 host services can advertise themselves via a simple **Host Configuration**. The following fields are needed by various TR-12 APIs.

serviceld: The pairing API requires this param. A host service shall reject pairing requests that do not provide a matching value.

serviceName: Describes the service name (friendly name) and may be used to inform the user to which service is being connected.

deviceTypes: A list of one or more the following: "SOURCE", "DESTINATION", "BOTH". A service may reject a pairing request for unsupported device types. A TR-12 service must support "SOURCE", "DESTINATION" or "BOTH".

pairingUrl: A complete, public REST endpoint that implements the *GetPairingCode* API.

authUrl: A complete, public REST endpoint that implements the *AuthenticatePairingCode* API.

thumbnailMaxSizeKB: Maximum size accepted for supporting thumbnail subscriptions: See Subscriptions

logFileMaxSizeKB Maximum size accepted for supporting log upload subscriptions: See Subscriptions

6 Pairing

Pairing is the process by which a client requests and receives the necessary credentials to authenticate with a TR-12 host. The client is responsible for securely persisting these credentials. Once successfully paired, a client can initiate and maintain MQTT connections until the credentials expire or are revoked by the host or the client. Paired clients can automatically reconnect after disconnections, power cycles or network interruptions without requiring an operator intervention to re-pair the device.

A TR-12 host shall provide the following HTTPS public REST APIs to handle pairing, the URLs for which are identified in the Host Configuration document described above. See Appendix *HostServiceApi: Pair and Authenticate Request*.

6.1 The pairing process

The client shall make Pair and Auth requests when credentials are not available for the given host. The client shall make a single *CreatePairingCode* request and only proceed to make Auth requests if the *CreatePairingResponse* returns success via the *CreatePairingCodeSuccessData*

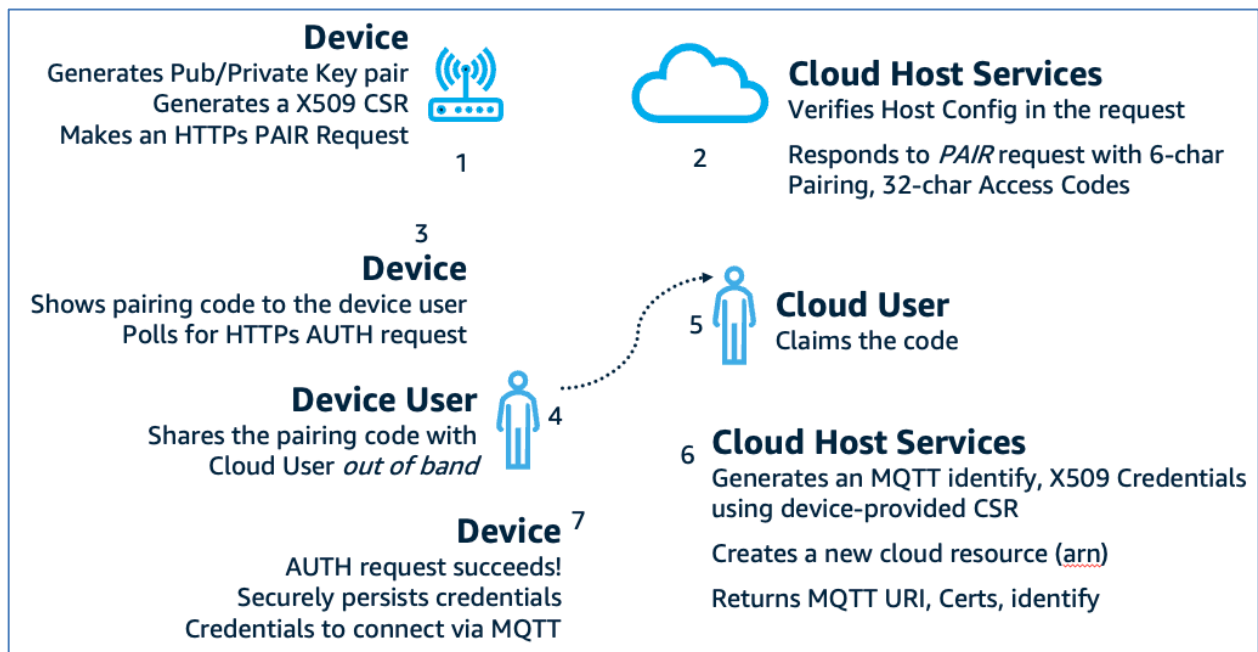
The host shall reject Pair API request whose *CreatePairingCodeRequest*: 1) Provides an unsupported *deviceType* 2) Incompatible *ProtocolVersion* 3) *Mismatched HostId*

The client shall make Authenticate requests at less than the Throttle Rate (see below) until the *AuthenticatePairingCodeResponse:AuthStatus* == 'CLAIMED'.

On successful *AuthenticatePairingCodeResponse*, the client shall persist the *AuthenticatePairingCodeResponse* payload that includes among other parameters the X509

credentials, HostSettings, and MQTT URI (endpoint) that are needed to connect to the host service. The client shall immediately MQTT connect and Register with the host to inform the host service that the pairing process is complete.

Fig 2 shows the pairing process steps initiated by the client Device on initial connection if no credentials corresponding to the given host are available. After successful pairing, the device persists the credentials. The credentials are used to connect to the host service via MQTT.



Pairing Security Features (Informative)

The pairing process includes several built-in security measures to protect against unauthorized device claims and brute-force attacks:

- **Rate Limiting (TPS):** The host service will implement **Transactions Per Second (TPS)** to prevent denial-of-service (DoS) and brute-force attempts.
- **Temporal Validity:** Pairing codes are only valid for a fixed window of time.
- **Access Code Entropy:** a **32-character access code**. The high entropy of this code ensures that the probability of spoofing within the expiration window is low.
- **Mitigation of Malicious Claims:** TPS limits "Host-side claim spoofing"
- **Transport Security:** **HTTPS** secures pairing request/responses payloads in transit.
- **Device Cert:** is generated by the TR-12 host using the client-provided **X509 Certificate Signing Request (CSR)**. Client MQTT connection requires the device private key and CSR.
- **Rotation:** x509 credentials are periodically updated via the TR-12 API once connected.

6.2 Throttling

- *Pair*: The client shall not exceed 1 request per minute.
- *Authenticate*: The client shall not exceed 1 request every 3 seconds.

6.3 Pairing Models

The following params described here appear in the Appendix related to:
CreatePairingCodeRequest, *CreatePairingCodeResponse*, *AuthenticatePairingCodeRequest* and *AuthenticatePairingCodeResponse*

- **deviceType**: must be one of "SOURCE", "DESTINATION" or "BOTH"
- **hostId**: The client provides a string describing the device type or unique instance
- **certificateSigningRequest**: The client provides an X509 Certificate Signing Request to the *PairRequest*
- **caCert**: The host Certificate Authority CA Certificate in PEM format <string>.
- **deviceCert**: : X509 Device Certificate generated by the host using the client provided CSR. On authorization, the host shall use the client-provided CSR and the hosts chosen Certificate Authority (CA) to generate an X509 device certificate.
- **version**: The client must identify the TR-12 SMITHY: *ProtocolVersion*: A host service shall reject pairing requests for unsupported versions. A host service must be backward compatible with all SMITHY model versions below the host's currently supported version.

7 MQTT Communication

7.1 Overview (Informative)

Protocol Overview MQTT is a lightweight, publish-subscribe (pub-sub) messaging protocol designed for secure, client-initiated connections to a central broker (the host service). By utilizing TLS and host-provided credentials, MQTT excels in IoT environments, maintaining "always-on" bidirectional communication through a persistent TCP connection. This architecture allows devices behind restrictive firewalls to receive real-time commands and transmit updates instantly. Its minimal header overhead ensures that low-latency payloads are delivered efficiently, even over unreliable or bandwidth-constrained networks.

7.2 MQTT Client: Settings

Both the client/host MQTT setting shall use the following the **MQTT v5.0** protocol encrypted via TLS using the credentials obtained during the pairing process.

The *AuthenticatePairingCodeResponse:HostSettings:iotProtocolName* parameter describes the ALPN protocol string. The client shall configure the IOT Client TLS settings using this string identifier.

*Note: **ALPN** stands for Application-Layer Protocol Negotiation. It is a TLS (Transport Layer Security) extension that allows the client and server to negotiate which application protocol they are going to use during the initial TLS handshake, before any actual data is sent. Typically, secure MQTT runs over port 8883. However, strict corporate firewalls often block port 8883, only allowing standard web traffic through ports 80 (HTTP) and 443 (HTTPS). ALPN solves this problem. It allows your MQTT client to connect to the broker on port 443*

7.3 MQTT Client: Processing a Configuration change.

Configuration Updates: Upon receipt of a configuration update, the client shall immediately apply the requested configuration to all relevant parameters

ConfigurationId: DeviceConfiguration includes a device and channel level configurationId (string) param. Hosts shall up this param why any parameter has changed. Clients obtaining a complete Configuration shall only apply device-level and channel-level params when the associated updateld has changed. By separating configurationIds between devices and channels, perturbation and processing load is applied only where needed,

Desired/Actual Configuration: Clients shall report actual configuration via *report_actual_pubReportActualConfigurationTopic* Topic immediately on updating any desired configuration. The client shall obtain actual state from the underlying device and not simply reflect the desired configuration. Clients shall reflect the desired configurationId at the associated device-level and channel-level.

Health: DeviceConfiguration: Health may be updated by the client for any device-level or channel-level problem. Clients should mark as HEALTHY unless otherwise and update the model params. Cloud users are thus informed about any client side issues that might otherwise present desired configuration from being applied.

Flow is: Get updated configuration, walk the configuration model->Check configurationIds -> apply changes to the device-> get device configuration state -> Report actual configuration (update Health and reflect relevant configurationIds)

7.4 MQTT Client: Registration

Client Registration: Registration payload is the primary mechanism by which the client defines its capabilities to the host.

- **On Connect:** The client shall publish the Registration payload immediately upon connecting to the MQTT broker on the *pubReportRegistrationTopic*
- **Host Logic:** The host service shall ignore all other MQTT requests from the client until this *Registration* payload is successfully received and processed.
- **Re-Registration:** The client may send an updated Registration payload at any time while connected provided the ONLY change is to: **DeviceRegistration**. Channel.profiles.
 - The host service can thus be informed about changes to the available profiles that a device use may have made since the device was connected.
 - The host service shall detect and apply ONLY Channel.profiles
 - The host service may update the Configuration.Channel.profile if needed to reflect changes to the Registered Channel.profiles.

7.5 MQTT Client: Reporting Status

Once connected and the client has published a registration payload, the client may provide telemetry to the host via a valid *DeviceStatus* payload model on the *pubReportStatusTopic*. See *Appendix: DeviceStatus*.

7.6 MQTT Client: Certificate Rotation

Rotation is the process by which a client replaces the certificates obtained during the initial pairing or a previous rotation. This mechanism allows the host to maintain security standards without requiring manual operator intervention at the device level.

Expiration Policy: The host shall determine the validity period of credentials and make rotation requests to the client they deem appropriate to enable persistent client access. **Note:** *Given an example expiration of 12 months, a host may elect to rotate credentials every month to ensure that a device can be offline up to 11 months and still be able to reconnect without re-pairing. A device must of course be online to process credential rotation. Rotating too often creates added client processing burden as well as a transient offline state, so while possible to rotate frequently, it's not recommended to rotate credentials more than necessary.*

Client Execution: Upon receiving a *CertRotate* message, the client shall immediately replace its local credentials, perform an MQTT disconnect, and re-establish the connection using the updated certificates. The client shall ignore *CertRotate* requests containing a duplicate payload.

7.7 MQTT Client: Deprovision

Deprovisioning is the process by which a client's lifecycle ends and subsequent reconnect requires (re)pairing. Both the host and client may initiate a deprovision. To avoid leaving a client in an ambiguous state when it has been deprovisioned while offline (not connected to the MQTT host)—where it cannot distinguish between a revoked credential and a network outage—the TR-12 deprovisioning process is designed to be transactional.

Client Requirements:

Service-Side deprovision: This is a deprovision initiated by the host service. The client, upon receiving a message on the *subDeprovisionTopic*, shall:

Client-Side Deprovision: This is a deprovision initiated by the device or device user. The client shall:

1. Publish an acknowledgment message to the *pubDeprovisionTopic*.
2. Immediately disconnect from the MQTT broker.
3. Delete all associated persistent credentials from the local file system.

Host Requirements:

Finality: Only after the host receives the client's **acknowledgment** message, shall the host permanently delete the credentials and associated resources.

Credential Grace Period. The host may delete the credentials after a period equal to 3x the default credential expiration if there was no client's **acknowledgment** message received.

8 Subscriptions

TR-12 provides a mechanism where a host service may request delivery of large payloads via a subscription. A subscription is a request to provide a given client payload that include a frequency, max-size and expiration to a given REST Put endpoint. TR-12 describes subscriptions for Thumbnails and Logs.

8.1 Overview (informative)

MQTT payload size and service operating cost limitations may not be suited for large payloads. As such, host-service operators will want control over when and how these large payloads are transmitted and paid for.

TR-12 defines a process for requesting clients send large, frequent payloads called **Subscriptions**. Subscriptions are an MQTT request, with delivery accomplished via the client making a REST PUT request to a host-provided, temporary secure endpoint defined by the: *ThumbnailRequest:remotePath*

The host service will generally use a pre-signed URL to a persistent host storage service valid from 5 to 30 minutes. The host service may even elect to require the user requesting the subscribed file to provide the destination resource URL and necessary access permissions to defer handling and storage costs to the user. *Note: all major cloud services offer pre-signed URLs for secure temporary write access to secure cloud storage via a REST endpoint.*

8.2 Thumbnail Subscription

- **Thumbnails are mapped to channels.** The thumbnail subscription will request a thumbnail on a per-channel-id basis. See: *ThumbnailSubscription* See model for required params.
- **JPEG and PNG are supported.** The client shall ensure that delivered images files include the following suffix: .jpg .JPEG .JPG .png .PNG.
- **Request:** the host may request thumbnails via the *subThumbnailSubscriptionTopic*. See model for required params.

- **Delivery:** The client shall deliver images according to the subscription request until the request has expired.

9 Nominal Operation Overview (informative)

9.1 Connect and Respond Loop (Informative)

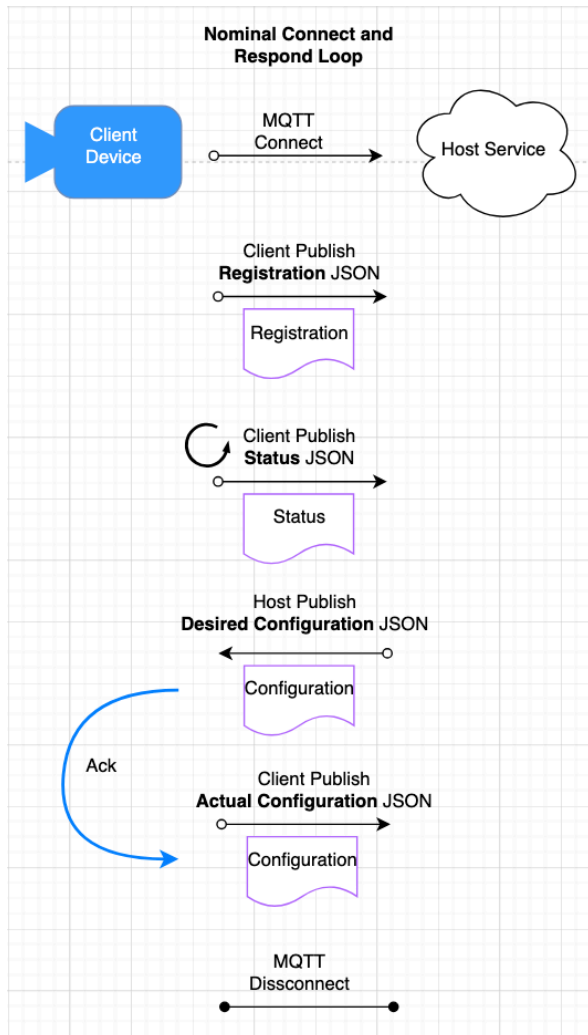


Fig3 shows a client connecting and then immediately publishing the **Registration** JSON. While connected, the client periodically updates status by publishing a **Status** JSON. The host service publishes a desired **Configuration** JSON request. The client device immediately updates its internal settings to comply then immediately publishes the actual **Configuration** JSON. This cycle continues until the device disconnects.

9.2 Subscription Overview (Informative)

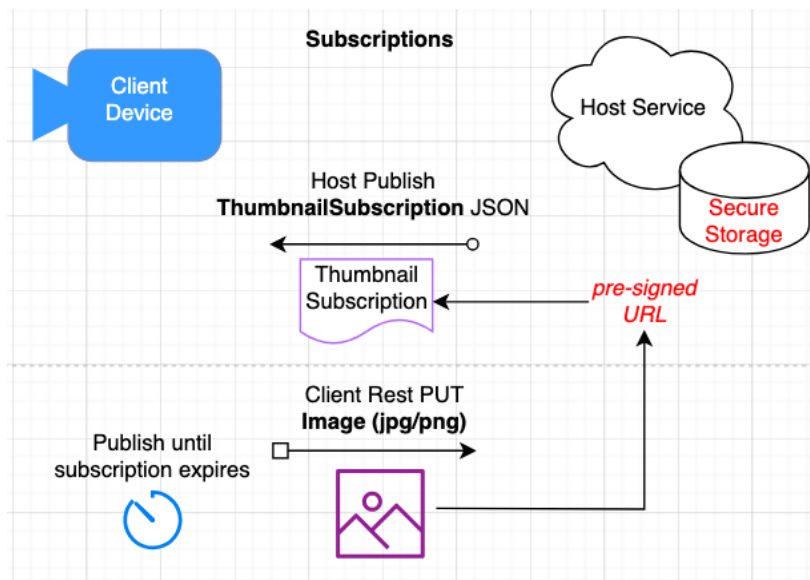


Fig4 shows a host requesting the client send thumbnail images periodically until the subscription expires to the given pre-signed, secure, temporary URL. The client repeatedly will upload images to the pre-signed URL via REST PUT.

9.3 Lifecycle Deprovision Overview (Informative)

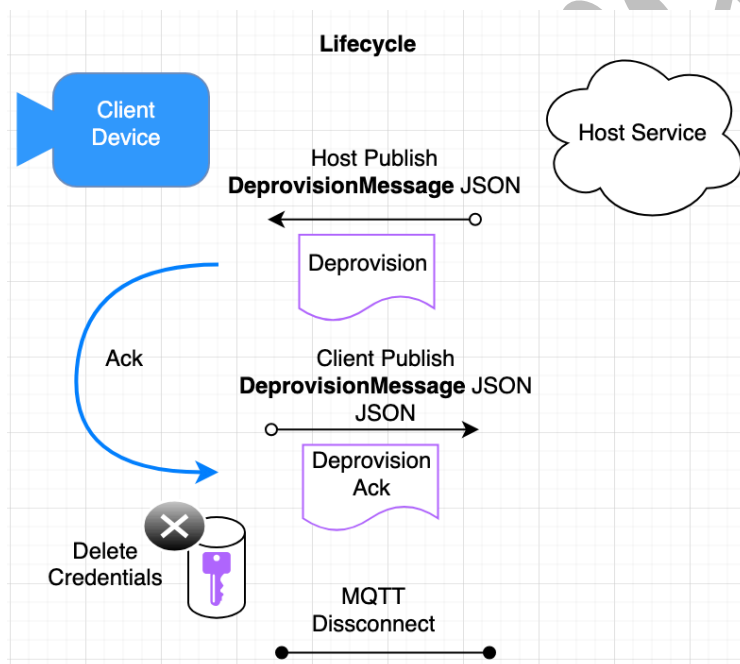


Fig5 Shows a host deprovisioning the client (Host Publish) and the Client acknowledging the request. Alternatively, the client can initiate a deprovision by publishing the same acknowledge message without a request from the host service. The client finally deletes credentials for this host.

10 The TR-12 Model (informative)

The TR-12 Model describes the client-service MQTT and HTTPS request/response contract. All client-service behaviors are built around these models. They enable common, versioned and validate-able message payloads and responses.

Why SMITHY: While SMITHY was created by AWS to manage its own massive ecosystem, it was released as an open-source project and has seen significant adoption by other large tech companies and open-source frameworks. Because SMITHY is protocol-agnostic (it doesn't force you into REST/HTTP like OpenAPI does), it is highly attractive to teams building complex microservices, RPC frameworks, or multi-language environments. SMITHY also provides a code generation mechanism that also translates into OpenAPI models, then creates programmatic class models and APIs in most languages. OpenAPI is fundamentally designed to describe HTTP REST APIs, often functioning as a documentation tool first and a code-generation tool second. Smithy was built from the ground up for code generation first. It treats the API as an abstract, mathematical model that can be compiled down into an HTTP REST server, a gRPC service, an MQTT stream, or a local CLI tool without having to rewrite the core business definitions.

Note: *OpenAPI models are automatically generated and are available in the open-source TR-12 SDK at: `../build/smithy/source/openapi`.*

Integrating the TR-12 model into a client program is accomplished by building the desired model in the desired language and calling a handful of HTTP and MQTT APIs. The generated models handle (de)serialization and validation.

10.1 The TR-12 Model Version

A critically important element of the TR12 Models is the Version. A client and host must have a common communication model for all messages.

<https://github.com/vsf-tv/TR-12-Models/>

The **common.smithy** includes a **ProtocolVersion** that must be supplied in the `CreatePairingCode` request model. This allows a host service to reject connections from devices with a mismatched TR-12 Version. This is available in the generated code model so clients have access via a static class accessor: `tr12models.ProtocolVersion`

```
structure ProtocolVersion {
    @default("2.0.1")
    version: String
}
```

Permitted TR-12-Model Changes: Once the TR12 draft is approved and formally committed, only non-breaking changes are permitted to preserve backward compatibility. This concept is a

widely used in large scale cloud services where clients written against one API are not affected by API changes. **Version updates may not:** add new required fields, change param names, types, remove items from enums, or remove fields. New fields must never be required and if supplied as optional can be safely and silently ignored when not supported by older version with no behavior change to in the underlying service. Basically ONLY new, optional items can be added.

TR-12 Protocol Versioning (Semantics)

V(major).(minor).(patch). le: V3.0.1

- MAJOR — Breaking change: removed/renamed field, type change, removed enum value, new required field, handshake change. Old clients/servers will fail. **NOT ALLOWED AFTER DRAFT TR12 becomes ratified.**

- MINOR — Additive, backwards compatible: new optional field, new enum value, new operation, new transport protocol variant. Old code ignores unknowns.

- PATCH — No wire format change: documentation, comments.

A host serve may reject pairing requests from a client that has an unsupported version, though this is not strictly a problem provided the versioning protocol has been applied

10.2 The TR-12 Models

Registration, Configuration, and Status models are formally defined in the Appendices via **SMITHY** specifications. These models act as a contract between the client and the host, ensuring that both sides have a shared understanding of the device's capabilities and data structures.

The Registration Process Upon initial connection, the client informs the host of its specific system capabilities. This registration payload includes:

System Capabilities: Specific configuration parameters on the local system.

Channel Architecture: The number of available media channels.

Per-Channel Settings: Specific configuration parameters available for each individual channel.

10.3 Registration Model

Dynamic UI Generation: The host can auto-generate a web-based management console. Using the types defined in the registration (e.g. integer, and ENUM string), the host renders "type-appropriate" UI widgets (such as toggles, sliders, entry fields, and pull downs widgets) without requiring device-specific front-end development.

Standard Settings: field-value pairs whose values can be **enums** or **ranges**. These are used to precisely configure device parameters.

Profiles: A profile is an ID, Friendly Name and description. These are used to select a 'composite' setting that may affect any number of actual channel settings.

Registration: The following is a partial Registration payload where *standardSettings* are defined as *enums* and *ranges* as well as *Profiles*. See Appendix: *Registration* for the complete model.

standardSettings: Example.

```
"name": "max_bitrate",
"description": "Maximum data rate limit (Kbps).",
"id": "MB01",
"ranges": {
  "min": 5000,
  "max": 50000,
  "defaultValue": 10000
}
```

Or

```
"name": "resolution",
"description": "Output video dimensions.",
"id": "RS01",
"enums": {
  "values": [
    "1920x1080",
    "1280x720",
    "640x480"
  ],
  "defaultValue": "1920x1080"
}
```

Profiles: Example:

```
{
  "name": "h265_4k",
  "id": "h265k",
  "description": "H265, 60fps, 40mps, 4k, 3s gop"
```

} ,

The host service shall validate Configuration payloads range and enum values against the Registration payload. **For example**, a Configuration payload where the MB01=50001 is invalid due to the range setting maximum of 50000 for that param.

10.4 Configuration Model

Configuration is formally defined in the Appendices. The Configuration payload is dependent on the **Registration** payload; the host service shall use the registration elements to validate configuration requests. The host shall ensure the Configuration payload contains only params defined in the Registration and conforms to defined *enums* and *ranges* values and or Profiles. **Configuration payload for a given param advertised in the Registration shall use the Registration.Id.** Registration.Name is merely for information, readability.

10.5 Status Model

System Status: Global health metrics (e.g., CPU load, memory usage, or connectivity strength).

Per-Channel Status: A structured list containing the **ID**, **Info**, and **Value** for each active media or data path.

System Status Example:

```
{
  "name": "cpu",
  "value": "41%",
  "description": "Current CPU utilization."
}
```

Channel Status Example

```
{
  "name": "bitrate-bps",
  "value": "0",
  "description": "Actual bitrate."
}
```

Status shall refer only to channels described in the Registration.

Appendix A SMITHY Models:

See: <https://github.com/vsf-tv/TR-12-Models>